DocHumanizer: Product Plan and Development Manual

Oct 26, 2026 · @tafadzwa usayi

1. Vision and positioning

DocHumanizer is a single-purpose web platform: upload a Word document, see which lines, sentences and paragraphs read as machine-written, and rewrite any of them in one click, without the file's formatting, tables, citations or equations changing. It does this one job better than any general writing tool, and it is not a chat assistant, a plagiarism checker or a general editor.

Product promise: "Open your .docx, see what reads as AI, fix it line by line, export a file that looks exactly like the one you uploaded."

Three pillars

1. **Fidelity.** The original .docx stays the source of truth. Only the sentences a user accepts are patched; everything else stays byte-for-byte identical.
2. **Line-level control.** Detection and rewriting work at the line, sentence and paragraph level, with suggestions the user chooses from, never silent bulk replacement.
3. **Platform intelligence.** The product learns each writer's voice, spots misuse of accounts, and improves its own models from consented feedback, independent of any single detector.

Integrity stance. AI detection is probabilistic and produces false positives, especially for non-native English writers and formal academic prose. The product therefore shows confidence ranges, never verdicts, never claims to be "undetectable", and is marketed as a tool for improving clarity and restoring a writer's own voice. This protects users from being wrongly accused, keeps the brand credible, and ages better than a promise to beat specific detectors, which change every few months.

Scope guard (not in v1): plagiarism matching, citation generation, a chat interface, real-time co-editing, and non-Word formats (PDF and Google Docs import come later).

2. Users and personas

Three account types drive every screen, limit and price in this document.

Persona	Who they are	Core job	Plan
Individual writer	Student, researcher or professional with their own papers	Fix my own document so it sounds like me	Individual, $5 per month
Edu Professional	Tutor, lecturer, editor, writing-centre staff, proofreader	Review and improve documents for several other people	Edu Pro, $15 per month
Workspace admin (later)	Department or institution lead	Manage seats, usage and policy for a group	Institution, custom
Platform operator (internal)	Our support and trust staff	Handle abuse, billing and model quality	Internal console

Individual writer, jobs to be done

- Upload a document and see at a glance where it reads machine-like.
- Rewrite one sentence, or auto-rearrange a paragraph, and keep the file's layout.
- Keep my own writing voice across every paper.
- Export a clean file, optionally with changes as tracked changes.

Edu Professional, jobs to be done

- Manage many client or student documents in folders, each with its own history.
- Apply a house style (APA, Harvard, department template) across documents.
- Share a read-only review link, or return an edited file to the document's owner.
- See my own throughput and turnaround time.

Why two tiers exist. The cheaper plan is licensed for one person's own writing. Handling other people's documents is a professional use, priced and monitored differently (see sections 9 and 10).

3. Feature catalogue

MVP is the smallest release that proves the core loop (upload, detect, rewrite, export with fidelity). V1 adds the commercial layer. V2 adds scale and depth.

Module	Feature	Release
Documents	Upload .docx and .dotx, drag and drop, up to 50 MB	MVP
Documents	Structure map: headings, paragraphs, tables, lists, footnotes, headers and footers, images	MVP
Documents	Protected atoms: equations, citations, tables, images, fields, cross-references skipped by AI	MVP
Documents	Equations shown and edited as LaTeX, saved back as native Word equations	MVP
Documents	Tracked-change export, or clean export	MVP
Documents	Version history with restore, per document	V1
Documents	Visual diff of original vs edited render	V1
Detection	AI-likelihood score per line, sentence and paragraph, with confidence band	MVP
Detection	Colour heat map over the document, with threshold slider	MVP
Detection	Document-level summary score with plain-language explanation	MVP
Detection	Reason chips per flagged sentence (uniform rhythm, generic phrasing, stock transitions)	V1
Detection	Re-score after every edit	MVP
Rewrite	Click a sentence, get 3 suggestions, apply one	MVP
Rewrite	Auto-rearrange: one click picks the best rewrite for a sentence or paragraph	MVP
Rewrite	Bulk "fix all above threshold" with a review queue	V1
Rewrite	Controls: tone, formality, length, keep terminology, keep numbers	V1
Rewrite	Meaning-preservation check shown as a badge on each suggestion	MVP
Rewrite	Undo, redo and compare original vs rewrite	MVP

Module	Feature	Release
Voice	Personal voice profile learned from the user's own samples	V1
Formatting	Style profiles (APA, Harvard, MLA, custom) mapped to real Word styles	V1
Formatting	Formatting check with one-click fixes and before/after preview	V1
Editor	Word-like editor with click-to-select sentences and floating action menu	MVP
Editor	Side panel for suggestions, reasons and history	MVP
Accounts	Email and Google sign-in, plans, invoices	MVP
Accounts	Individual and Edu Pro plans, daily limits, usage meter	MVP
Accounts	Folders and client labels for Edu Pro	V1
Accounts	Shareable review link (read-only, expiring)	V1
Trust	Account-sharing and misuse detection, enforcement ladder	V1
Admin	Operator console: users, billing, abuse queue, model quality	V1
Institution	Seats, SSO, policy controls, pooled usage	V2
Import	PDF and Google Docs import	V2
API	Public API for batch processing	V2

4. Document engine

The document engine is the riskiest and most valuable component: a rewrite that breaks a table or loses a footnote ends the product's credibility. The rule is that the uploaded .docx is never converted to HTML and back. It is opened as the package it is, and edits are patched into it.

4.1 Parsing

A .docx is a zip of XML parts (document.xml, styles.xml, numbering.xml, footnotes.xml, headers, footers, media). The parser reads these into a structure map:

1. Walk document.xml in order and create a node for every paragraph, table, cell, list item, field, equation, image and section break.
2. Split each paragraph into **runs** (chunks with their own bold, italic, font, hyperlink or comment anchor). One sentence often spans several runs.
3. Segment each paragraph into sentences with a sentence splitter that is aware of abbreviations, citations like (Smith, 2021), decimals and equations.
4. Give every paragraph, sentence and run a **stable ID** and record its exact XML location and character offsets.

4.2 Sentence-to-run mapping and patching

When a user accepts a rewrite for sentence S:

1. Find the runs S covers, and the formatting at its start.
2. Diff the old and new text at word level, so unchanged words keep their original run formatting and only changed words take new text.
3. Preserve inline anchors (hyperlinks, comment ranges, bookmarks, tracked changes by others) by refusing the rewrite of any sentence whose anchor would be destroyed, and offering to rewrite around the anchor instead.
4. Write the new text into the runs, leaving every other XML node untouched.
5. Re-segment that paragraph and update IDs.

4.3 Protected atoms

Before detection or rewriting, the engine replaces anything the AI must not touch with an opaque token, then restores it exactly on export.

Atom	Handling
Equations (OMML)	Token in text; editable separately as LaTeX
Citations and references	Token; optional "check unchanged" guard
Tables	Cell text is detected and editable, structure is locked
Images, charts, shapes	Locked, never sent to a model

Atom	Handling
Fields (page numbers, TOC, cross-references)	Locked
Quoted text (block quotes, direct quotations)	Locked by default, with a toggle
Numbers, dates, units	Extracted and verified present, unchanged in every rewrite

4.4 Equations and LaTeX

Word stores equations as OMML. The chain is OMML to MathML to LaTeX for display and editing, and LaTeX back to MathML to OMML for export. Pandoc and the Microsoft OMML-to-MathML stylesheet cover most of the work; edge cases (matrices, multi-line alignment, custom operators) are covered by a regression suite of real documents. The editor renders formulas with KaTeX and opens a LaTeX edit box with live preview when clicked. If a formula fails to round-trip, the original OMML is kept untouched and the user sees a "locked" badge.

4.5 Formatting profiles

A profile is a set of rules mapped to real Word styles (Heading 1, Body Text, Caption, Block Quote), not direct formatting, so the output stays clean and editable in Word. Rules cover fonts, sizes, spacing, margins, heading numbering, caption placement, reference style and page numbering. The engine reads the document's actual styles, lists deviations from the chosen profile, and offers a before/after preview with one-click apply. A custom profile can be built from an uploaded template (.dotx).

4.6 Vision and layout QA

The XML is the ground truth for structure, so vision models are used for verification and for the hard cases:

- Render the original and the edited document to page images with LibreOffice, then compare them. Any unexpected shift is flagged before export.
- For documents that are really scanned images or have faked structure (spaces used as indents, text boxes used as columns), a vision model proposes the intended structure and the user confirms it.
- Detect layout damage the XML does not show: a table overflowing a page, an orphaned heading, or a missing image.

4.7 Export

Two modes: **clean** (edits applied) or **tracked** (every accepted edit written as a Word tracked change attributed to the user, so a reviewer can accept or reject in Word). A fidelity report lists what changed and confirms what did not.

4.8 Fidelity test suite

A library of 500 or more real-world documents (theses, reports with embedded Excel charts, legal templates, documents with comments, right-to-left text) is run on every release. A release fails if any untouched paragraph's XML differs from the original, or if the visual diff exceeds a pixel threshold on untouched pages.

5. Detection engine

Detection is an estimate, not a fact, and the design reflects that: it scores text, shows a confidence band, and is tuned to avoid false accusations before it is tuned to catch more.

5.1 Model design

The detector is an ensemble. Each member sees the text from a different angle, and a small calibration layer combines them.

1. **Fine-tuned transformer classifier.** An encoder such as DeBERTa-v3 or RoBERTa, fine-tuned on paired human and machine text.
2. **Statistical features from a reference language model.** Per-token perplexity, burstiness (how much perplexity varies across a passage), and rank-based features. A small open model such as Qwen or Llama-class 1 to 8B parameters supplies the probabilities.
3. **Stylometric features.** Sentence-length variance, punctuation rhythm, vocabulary richness, transition-word density, hedging and list-like structure.
4. **Context smoother.** A sequence model that takes the scores of neighbouring sentences, because single sentences are noisy and paragraph context is far more reliable.

5.2 Granularity

Level	Reliability	How it is shown
Document	Highest	Summary score with plain-language explanation
Paragraph	High	Colour band behind the paragraph

Level	Reliability	How it is shown
Sentence	Medium	Underline with hover score and reason chips
Line (visual line)	Low	Mapped from its sentence; never scored on its own

Very short sentences (fewer than about eight words), titles, headings, lists and quotations are not scored on their own, because there is too little signal.

5.3 Calibration and false-positive policy

- Scores are calibrated so "70 percent" means roughly 70 percent of such text in the validation set was machine-written.
- Target: a false-positive rate under 1 percent on human text at the highest-confidence setting, measured separately on native, non-native, formal-academic and technical writers.
- The UI shows three bands (likely human, uncertain, likely machine) instead of a single verdict, and uncertain is a real, large band.
- If the document is mostly formulas, tables or references, the engine states that it has too little prose to score.

5.4 Training data

- Human text: public-domain and openly licensed essays, theses, journal articles, forum writing and consented user uploads, across disciplines and English varieties.
- Machine text: generated from many open and API models, at several temperatures and prompts, and in several styles (essay, report, summary).
- Hard negatives: human text that has been edited by grammar tools, and machine text that has been paraphrased or lightly edited by people.
- A fresh batch of machine text from newly released models is added on a regular cadence, because detectors age quickly.

5.5 Evaluation

A frozen evaluation set is never used for training. Release gates: false-positive rate by writer group, AUROC, calibration error, and performance on paraphrased and lightly edited machine text. Every model version is stored with its metrics, and a regression on any gate blocks release.

6. Rewrite and suggestion engine

The rewrite engine produces alternatives the user chooses from. It is built to keep meaning, facts and voice intact, and to make the text read like the person who wrote it, not like a generic "humanized" style.

6.1 Pipeline for one sentence

1. **Context window.** Send the target sentence plus its paragraph, headings and a short document summary, so the rewrite fits the argument.
2. **Constraints.** Lock protected atoms, numbers, named entities, terminology and citations as must-keep items.
3. **Generate.** An instruction-tuned open model produces several candidates with different strategies: restructure, vary rhythm, replace generic phrasing, split or merge.
4. **Verify meaning.** Check each candidate with sentence-embedding similarity plus a natural-language-inference model (must entail the original, and the original must entail it).
5. **Verify constraints.** Every locked number, entity and citation must still be present and unchanged.
6. **Re-score.** Run detection on the candidate in its paragraph context, and drop candidates that do not improve readability or the score.
7. **Rank and present.** Show the top three with a meaning badge, a score change and a plain-language note on what changed.

6.2 Auto-rearrange

One click picks the top-ranked candidate for a sentence, or for a whole paragraph, and applies it with undo. Paragraph mode rewrites sentence by sentence while checking transitions, so the paragraph still flows. Bulk mode builds a **review queue** so nothing is applied silently.

6.3 User controls

Control	Options
Tone	Neutral, academic, conversational
Formality	Slider from plain to formal
Length	Shorter, same, longer
Strength	Light touch, moderate, full restructure

Control	Options
Keep	Terminology list, numbers, quoted text

6.4 Voice profile

A user uploads two or three of their own earlier documents. The system learns their typical sentence length, vocabulary, punctuation and connective phrases, and stores this as a style vector plus a short style guide. Rewrites are conditioned on it, so the output moves toward how that person actually writes. This is the strongest differentiator from generic rewriters, and it also gives better results than any detector-chasing approach.

6.4.1 Guardrails

- Never invent facts, citations or numbers. A rewrite that introduces new factual claims is rejected.
- Never rewrite locked text (quotes, equations, references).
- Show the original next to every suggestion, and keep a full edit history.
- Rate-limit bulk operations to prevent mass automated use (section 10).

7. Editor UI and screens

The editor feels like Word with a review layer on top: the page looks like the user's document, and the intelligence appears only when asked for or when a sentence is selected.

7.1 Screen map

Screen	Purpose
Landing and pricing	Explain the product, show plans, start sign-up
Sign-up and plan choice	Choose Individual or Edu Professional, verify email, add payment
Dashboard	Documents, usage, plan status (section 8)
Upload	Drag and drop, progress, structure scan result
Editor	Main workspace for one document
Compare	Original vs edited, side by side, with changes listed

Screen	Purpose
Export	Clean or tracked, with fidelity report
Voice profile	Add samples, view the learned style
Formatting profiles	Choose, build or import a style profile
Billing and account	Plan, invoices, devices, sessions

7.2 Editor layout

- **Centre:** the document, rendered page by page with the original fonts, tables, images and equations. Sentences carry a coloured underline by band; paragraphs carry a soft background by band.
- **Left rail:** outline (headings), page thumbnails, and a heat strip showing where flagged text clusters.
- **Right panel:** tabs for Suggestions, Score details, Formatting, History.
- **Top bar:** document name, save state, undo and redo, threshold slider, view toggles (heat map, original, changed), Export.
- **Floating menu on selection:** Rewrite, Suggest, Auto, Edit by hand, Lock, Ignore.

7.3 Core interactions

1. Click a sentence. It highlights, the right panel shows its score, reasons and three suggestions.
2. Hover a suggestion to preview it inline in the document; click Apply to commit it.
3. Click Auto to apply the top suggestion with an undo toast.
4. Select a whole paragraph and click Auto paragraph for a coherent rewrite.
5. Click an equation to open the LaTeX box with live KaTeX preview.
6. Press Lock on any sentence to protect it from rewriting and scoring.
7. Ignore marks a sentence as reviewed so it stops appearing in the flag count.

7.4 Keyboard and accessibility

J and K move between flagged sentences, A applies the top suggestion, U undoes, L locks, and the full list is in a shortcut sheet. Colour is never the only signal (bands also carry patterns and labels), the editor meets WCAG 2.2 AA, and everything is operable by keyboard and screen reader.

7.5 Performance targets

A 30-page document opens in under 4 seconds, detection streams in as paragraphs finish (first scores within 5 seconds), a single-sentence suggestion returns in under 3 seconds, and applying a change updates the view in under 100 ms.

8. Dashboards

There are three dashboards: one for Individual writers, one for Edu Professionals, and an internal operator console. Each lists its widgets and the functions behind them.

8.1 Individual dashboard

Widget	What it shows	Functions
Upload zone	Drag-and-drop area	Upload .docx, see scan progress
My papers	Cards for each paper with status and score	Open, rename, duplicate, archive, delete, restore from history
Paper status	Draft, in review, ready	Change status, add a due date
Score trend	Document score before and after edits, per paper	Compare versions
Usage meter	Documents and edits used today and this month	Upgrade prompt when near the limit
Voice profile card	Samples added, profile strength	Add or replace samples, reset profile
Recent activity	Last edits, exports and restores	Jump back into a paper
Plan and billing	Plan, renewal date, invoices	Change payment method, cancel, upgrade to Edu Pro
Tips	Short guidance based on the user's own flagged patterns	Dismiss, learn more

Paper rule on this dashboard. The Individual plan is for the account holder's own writing. The dashboard shows a papers counter and a small "Is this your own paper?" confirmation at upload, which is part of the licence and feeds the misuse model in section 10.

8.2 Edu Professional dashboard

Widget	What it shows	Functions
Clients and folders	A folder per client, student or class	Create, rename, merge, archive, bulk-move documents
Document board	Kanban of Uploaded, In review, Returned, Closed	Drag to change stage, assign labels, set deadlines
Queue and deadlines	Papers sorted by due date	Filter by client, overdue alerts
Throughput	Papers closed per day and week, average turnaround	Date-range filter, export to CSV
Quality snapshot	Average score drop per document, acceptance rate of suggestions	Per-client breakdown
Batch tools	Run scoring or formatting across a folder	Start batch, view the review queue
Share and return	Read-only review links, send edited file back	Set expiry, revoke, track opens
Style profiles library	House styles for each client or institution	Create, import .dotx, assign to folder
Usage and limits	Daily and monthly usage against Edu Pro limits	Request a higher limit
Billing	Plan, invoices, receipts per client for the user's own records	Download PDF, change card

8.3 Operator console (internal)

Area	What it shows	Functions
Overview	Active users, new sign-ups, revenue, conversion, churn	Date-range filter, segment by plan
Users	Searchable list with plan, usage and risk score	View detail, change plan, issue refund, suspend, impersonate with audit log

Area	What it shows	Functions
Abuse queue	Accounts flagged by the misuse model, with evidence	Review, warn, throttle, upgrade-required, suspend, dismiss
Billing	Payments, failures, disputes, invoices	Retry, refund, credit
Usage and cost	Model calls, GPU hours, cost per user and per plan	Alerts when cost per user exceeds revenue
Model quality	False-positive rate by writer group, suggestion acceptance, meaning-check failures	Compare model versions, roll back
Feedback review	Consented user corrections and thumbs ratings	Approve into training set, reject
Fidelity monitor	Export fidelity failures and visual-diff alerts	Open the failing document, assign to engineering
Content policy	Reports and blocked uploads	Review, action
Feature flags and limits	Plan limits, daily caps, rollouts	Edit limits, toggle features per cohort
Audit log	Every operator action	Filter, export

8.4 Notifications

In-app and email notifications cover: approaching a daily limit, a payment failure, a flagged account warning, an export ready, a review link opened, and a shared document returned.

9. Pricing, plans and limits

Two prices, built on one licence rule: a $5 plan covers a person's own papers, and a $15 plan covers working on other people's papers. The numbers below are starting proposals to be tested against real model cost (section 12).

9.1 Plans

	Pay per paper	Individual	Edu Professional
Price	$5 per paper, one time	$5 per month	$15 per month
Whose papers	The buyer's own	The account holder's own	Other people's papers (clients, students)
Papers	1 paper, unlimited edits on it for 30 days	Active-papers cap (proposed 3 active at a time)	Active-papers cap (proposed 40 active at a time)
Daily fair-use limit	Up to the paper's size cap	Daily word and suggestion cap (proposed 30,000 words scored and 300 suggestions per day)	Higher daily cap (proposed 150,000 words and 1,500 suggestions per day)
Folders and clients	No	No	Yes
Share and return links	No	Review link only	Yes
Voice profiles	No	One (own)	Several (one per client, with consent)
Batch tools	No	No	Yes
Support	Standard	Standard	Priority

9.2 Rules behind the limits

- **Edits on a paper are unlimited within the daily cap.** The cap exists to control GPU cost and automated misuse, not to ration normal work.
- **Active papers**, not lifetime papers, are counted. Archiving a finished paper frees a slot, so a student with one paper at a time is never blocked.
- Caps reset at midnight in the user's time zone.
- A user near a cap sees the meter, and can upgrade or buy a single paper at $5.
- Annual billing gets a discount (proposed two months free).
- A 3-day trial or one free scored page lets people see the product before paying.

9.3 Edu Pro verification and trust

Edu Pro is open to anyone who accepts the professional terms, because verifying every tutor or editor is costly and excludes freelancers. A user who signs up as Edu Pro states their use (tutor, lecturer, editor, writing centre). An institutional email or ID raises their trust level and unlocks higher caps, but is not required.

9.4 Billing mechanics

- Payments through a processor with international coverage (Stripe, with a local mobile-money or card option for markets where Stripe is limited, for example Paddle or Flutterwave).
- Subscriptions renew monthly, with a 3-day grace period after a failed payment.
- Upgrades are prorated immediately, downgrades take effect at the next renewal.
- Pay-per-paper purchases create a time-boxed licence (30 days) attached to that one document.
- Invoices and receipts are generated for every charge, and can carry a company name for Edu Pro users.

9.5 Questions for you to settle

1. Is "$5 per person or per paper" meant as two options (a $5 subscription for one's own papers, and a $5 single-paper purchase), as proposed above?
2. Should daily limits be hard caps (as proposed), or should the Individual plan be truly unlimited per day with only abuse throttling?
3. How many active papers should the Individual plan allow at one time?
4. Should Edu Pro users be allowed to resell the output to clients? The plan text assumes yes, as part of their service.

10. Account-sharing and misuse detection

The goal is fair pricing, not surveillance: an Individual account used for many different people's papers should be nudged to Edu Pro, and accounts shared between people should be spotted. No single signal triggers a penalty; a risk score built from several signals does, and a human reviews the serious cases.

10.1 Signals

Signal	What is checked	Why it matters
Authorship style	A stylometric fingerprint (sentence length, vocabulary, punctuation habits) is computed per paper and compared across the account	Several clearly different writers on one Individual account suggests other people's papers
Document metadata	Author, last-modified-by, company and creation tool fields in the .docx	Many different author names across papers
Title-page entities	Names, student numbers, course codes and institutions on the first page, found with entity extraction	Different names on different papers
Subject spread	Topics and course codes across papers	A tutor-like spread across unrelated courses and levels
Own-paper confirmation	The upload checkbox and the plan's paper cap	A user confirming "my own paper" for papers whose signals disagree
Upload and return pattern	Papers uploaded in bursts, quick exports, files sent on to outside email addresses via share links	Service behaviour rather than personal writing
Devices and sessions	Number of devices, locations, simultaneous sessions, impossible travel	Shared login credentials
Payment links	Same card, same billing name or same device across many accounts	One operator running several cheap accounts
Velocity	Papers and words per day against the plan's norm	Automated or resale use

10.2 Scoring

1. Each signal becomes a feature on a rolling 30-day window.

2. A gradient-boosted tree model (trained on labelled outcomes from reviewed cases) produces a risk score from 0 to 100, and a plain-language list of the top contributing reasons.
3. Rules sit on top for clear-cut cases: for example four or more distinct authors in a month on an Individual plan, or more than three simultaneous devices.
4. Before enough labelled data exists, rules and thresholds only are used, and reviewers label every flagged case to bootstrap the model.

10.3 Enforcement ladder

Level	Risk score	Action	User sees
0	Under 40	None	Nothing
1	40 to 59	Soft nudge	"Working on other people's papers? Edu Pro is built for that"
2	60 to 74	Confirmation and upgrade prompt	Must confirm ownership on each new paper, or upgrade to Edu Pro
3	75 to 89	Throttle	New uploads limited, the user is told why and how to fix it
4	90 and above, or repeated level 3	Human review, then hold or suspend	Email with the reason and an appeal link

Anyone upgrading to Edu Pro resets the account to level 0 with the higher caps. A suspended user keeps read access to export their existing documents.

10.4 Fairness, privacy and appeals

- The signals are disclosed in the terms of service and the privacy policy in plain language.
- Stylometric fingerprints are stored as numeric vectors, not as copies of the text, and are deleted with the account.
- Document text is not read by staff in the abuse queue unless the account holder opens a ticket.
- Every action at level 3 or above shows the user the reason and offers an appeal, answered within two working days.

- False positives are tracked as a metric (the share of flagged accounts later cleared on appeal) with a target under 5 percent.
- Legitimate cases are expected: a student editing a group project, or a parent helping a child. The reviewer playbook lists these, and the nudge language is courteous, not accusatory.

10.5 Abuse beyond account sharing

The same service covers credential sharing, free-trial farming (device and payment fingerprints), automated scraping through the API or UI (rate limits and bot checks), chargeback fraud, and uploading of illegal or malicious files (virus scan, macro stripping, file-type checks).

11. Platform intelligence (independent of AI detection)

The platform's value must not depend on one detector staying accurate. These intelligence layers keep working, and keep improving, whatever detectors the outside world uses.

Layer	What it does	Method
Writing quality score	Rates clarity, flow, repetition, specificity and structure, separate from the AI score	Small regression model on rubric-labelled text, plus rule checks
Voice match score	Shows how close a paragraph is to the user's own voice profile	Style-vector distance
Smart suggestions	Ranks suggestions by how likely this user is to accept them	Learned from each user's accept and reject history
Document health	Finds repeated phrases, overused transitions, inconsistent terms, long sentences	Linguistic rules and embeddings
Consistency guard	Flags changes of tense, spelling variety (US and UK), name and term spelling	Rules and entity tracking
Structure advice	Suggests paragraph splits, merges and reordering for flow	LLM with structure constraints

Layer	What it does	Method
Personalised tips	"You often start sentences with 'Moreover'" with an example from the user's own text	Pattern mining on the user's documents
Usage forecasting	Predicts daily and monthly load per plan to size GPU capacity	Time-series model
Churn and upgrade signals	Detects users who would benefit from Edu Pro, or are likely to cancel	Gradient-boosted model on behaviour
Quality monitoring	Watches meaning-check failures, suggestion acceptance and fidelity errors	Streaming metrics and alerts

11.1 The learning loop

1. Every suggestion shown is logged with its context, model version, user action (accept, edit then accept, reject) and score change.
2. Users can rate a suggestion with a thumbs up or down and optionally give a reason.
3. Only users who opt in contribute text to training; others contribute anonymous aggregate metrics only.
4. Reviewed, consented examples feed the next fine-tuning round for the rewriter and the quality models.
5. A new model version first runs in shadow mode (its outputs are logged but not shown), then to 5 percent of traffic, then to everyone, with automatic rollback if acceptance or meaning-check metrics drop.

11.2 Why this matters commercially

A detector can be copied. A document engine with high fidelity, a voice profile that improves with use, and an accepted-suggestion data set that only this product has are harder to copy. These layers also keep the product useful to writers whose text is never flagged at all.

12. AI model stack and fine-tuning plan

The plan uses open-source models throughout, so cost per user stays predictable and no vendor controls the product. Model families are named as examples; open-model rankings and licences change every few months, so re-check licence terms (commercial use, size limits) and current benchmarks before committing to a specific model.

12.1 Models by job

Job	Model type	Starting candidates	Size and hosting
Detection classifier	Encoder, fine-tuned	DeBERTa-v3, RoBERTa, ModernBERT	Small (under 1B), CPU or a shared GPU
Reference language model for perplexity	Decoder, used frozen	A small Qwen, Llama or Mistral-class model	1 to 8B, quantised, shared GPU
Rewriter	Instruction-tuned decoder, fine-tuned with LoRA	Qwen, Llama or Mistral-class instruct models	7 to 14B for MVP, larger for a premium tier
Meaning check (similarity)	Sentence embedding model	A sentence-transformers or BGE-class model	Small, CPU
Meaning check (entailment)	NLI cross-encoder	A DeBERTa-based NLI model	Small
Voice and style	Embedding and stylometric features	Sentence embedder plus handcrafted features	Small
Sentence and entity tools	NLP pipeline	spaCy or Stanza	CPU
Layout and scanned-document vision	Vision-language model	An open document-understanding model	Small to medium, GPU on demand
Misuse model	Gradient-boosted trees	XGBoost or LightGBM	CPU

12.2 Fine-tuning plan

Rewriter.

1. **Base data.** Build pairs of (machine-sounding sentence in context, human-sounding rewrite that keeps meaning), from three sources: licensed or open human-edited text; machine text that human editors revised; and synthetic pairs from a stronger model, filtered by the meaning and constraint checks in section 6.
2. **Supervised fine-tuning (LoRA or QLoRA)** on those pairs, with the instruction format including constraints, tone and voice-profile fields.
3. **Preference tuning.** Use real accept, edit and reject signals to train the model to prefer what users accept (DPO or similar), and to penalise meaning drift.
4. **Voice conditioning.** Train the model to follow a short style guide and a few example sentences supplied at request time, so the same model serves every user's voice without a separate model per person.
5. **Distillation.** After the larger rewriter is good, distil it into a smaller, faster model for the free-flowing everyday path, keeping the larger one for hard cases.

Detector.

1. Fine-tune the encoder on the human and machine corpus described in section 5.4.
2. Add adversarial rounds: paraphrase the machine text with the rewriter and other tools, and retrain so the detector is not blind to rewritten text.
3. Fit a calibration layer (isotonic regression or Platt scaling) per writer group.

12.3 Evaluation

Model	Key metrics
Detector	False-positive rate by writer group, AUROC, calibration error, robustness to paraphrase
Rewriter	Meaning-check pass rate, constraint pass rate, human preference ratings, user acceptance rate, rewrite latency
Voice	Held-out authorship identification accuracy, blind human match ratings
Misuse	Precision at each enforcement level, share of flags cleared on appeal

Human evaluation uses a panel of editors rating blind A and B comparisons on a fixed rubric (meaning kept, reads naturally, fits context) at every major model release.

12.4 MLOps

- Experiment tracking and model registry (MLflow or Weights and Biases), with every model tied to its data version and metrics.
- Data versioning (DVC or lakeFS) and a documented data card for every training set.
- Serving on vLLM or TGI for the decoders, ONNX or Triton for encoders, autoscaled on queue depth, with batching to keep GPU use high.
- Shadow, canary and rollback process as described in section 11.1.
- Cost dashboard that shows model cost per document and per plan, because the pricing in section 9 depends on it.

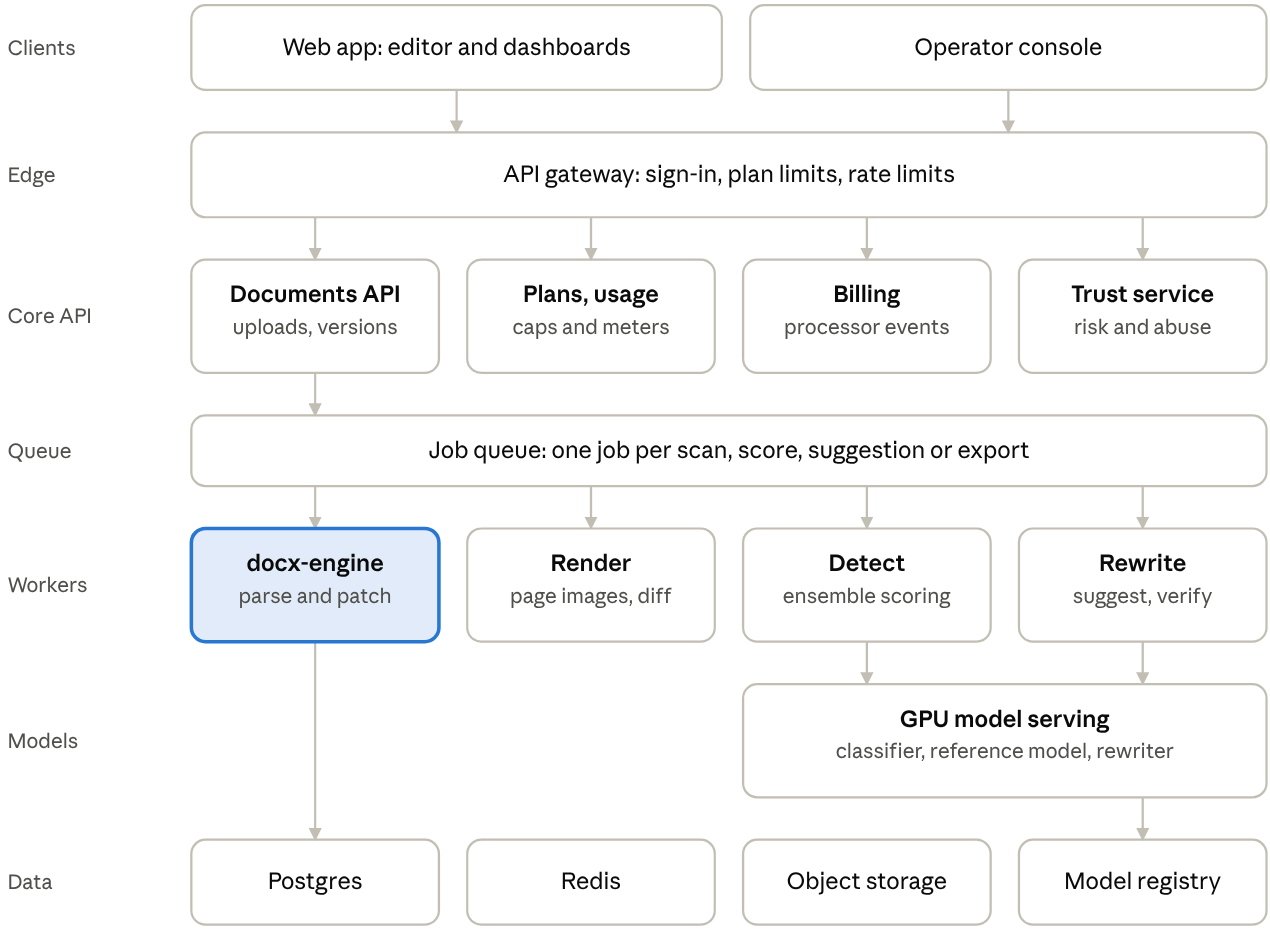
12.5 Data, rights and consent

Only licensed, openly licensed or consented data is used for training. User documents are never used for training unless the user opts in, and opted-in text has names and identifiers removed first. Training data lists are kept so that a deletion request can be honoured in the next retraining cycle.

13. System architecture

A Next.js web app talks to a stateless API; all document work runs as queued jobs on workers, and models are served from a shared GPU pool.

One gateway and one queue sit in front of every document job



platform architecture · 7 layers, 4 workers

The docx-engine is the only service that reads document text, so protected atoms are handled in one place. All services read and write the stores in the bottom row through their own service accounts.

13.1 Frontend

Part	Choice	Why
Framework	Next.js, React, TypeScript	Server rendering for marketing pages, client app for the editor
Editor	TipTap or ProseMirror with a schema that mirrors Word's structure	Fine control over sentences, marks and atoms

Part	Choice	Why
Page view	Virtualised page rendering from the render service and the structure map	Large documents stay smooth
Equations	KaTeX	Fast LaTeX rendering
Live updates	Server-sent events or WebSockets	Scores arrive paragraph by paragraph
State	A small client store plus server state cache	Optimistic apply and undo
Design system	Shared component package with accessible primitives	Consistent dashboards and editor

13.2 Backend

Part	Choice	Why
API	TypeScript (NestJS or Fastify), OpenAPI contract	Typed, easy to scale horizontally
Docx engine	Python with lxml and python-docx, with an Open XML SDK service as a fallback for hard cases	Direct XML control
Render	LibreOffice headless in sandboxed workers	Page images for visual diff and thumbnails
Queue	Redis-based job queue (BullMQ or Celery) with priorities per plan	Edu Pro jobs get priority; retries and dead-letter handling
Database	PostgreSQL with pgvector	Relational data plus style and embedding vectors
Object storage	S3-compatible, encrypted	Original and edited .docx files
Model serving	vLLM or TGI for decoders, Triton or ONNX for encoders	High throughput through batching
Auth	OIDC provider with email, Google sign-in and optional two-factor	Fewer security mistakes than rolling our own

Part	Choice	Why
Billing	Stripe, with a second processor for markets Stripe does not cover	Subscriptions, proration, invoices
Observability	OpenTelemetry, Prometheus, Grafana, Sentry	One trace per document

13.3 Life of a document

1. **Upload.** The file goes straight to object storage; the API creates the document row and a scan job.
2. **Scan.** The docx-engine builds the structure map, finds atoms, and segments sentences. The editor opens as soon as the map is ready.
3. **Detect.** Paragraphs are sent to the detection service in batches; scores stream to the editor.
4. **Suggest.** A click on a sentence creates a suggestion job; the rewrite service returns checked candidates.
5. **Apply.** The chosen text goes to the docx-engine, which patches the XML and records an event.
6. **Export.** The engine writes the .docx, the render service builds before and after images, and the fidelity report is produced.

13.4 Scaling and cost control

- API and workers are stateless and scale on queue depth; GPU workers scale on request latency and queue length.
- Detection runs once per paragraph per version and is cached; only changed paragraphs are re-scored.
- Suggestions are generated on demand, not for the whole document up front, which keeps cost proportional to user effort.
- Small distilled models handle the common path; the larger rewriter is used for hard cases.
- Plan caps (section 9) and the cost dashboard (section 8.3) keep cost per user under revenue per user.
- Hosting can start in one cloud region with rented GPUs, and add regions when data-residency or latency needs appear.

14. Data model

Postgres holds the relational data, object storage holds the files, and Redis holds sessions, queues and counters. These are the core tables.

Table	Key fields	Notes
users	id, email, name, plan_type, country, time_zone, trust_level, created_at	One row per person
subscriptions	id, user_id, plan, status, period_start, period_end, processor_ref	Mirrors the billing processor
paper_licences	id, user_id, document_id, expires_at	Pay-per-paper purchases
documents	id, owner_id, folder_id, title, status, source_object_key, current_version_id, owner_declared_own	The upload and its state
document_versions	id, document_id, version_no, object_key, created_by, reason	Full history for restore
nodes	id, document_id, version_id, type, parent_id, xml_path, order_no	Paragraph, table, run, atom
sentences	id, node_id, text, char_start, char_end, locked, ignored	Stable sentence IDs
scores	id, sentence_id, model_version, score, band, confidence, reasons_json	Per version and model
suggestions	id, sentence_id, model_version, text, meaning_score, score_delta, strategy	Shown to user
edits	id, sentence_id, suggestion_id, action, old_text, new_text, user_id, at	Accept, edit, reject, undo
atoms	id, document_id, type, token, original_xml_ref	Equations, citations, locked items
folders	id, owner_id, name, client_label, style_profile_id	Edu Pro organisation
style_profiles	id, owner_id, name, rules_json, source_template_key	Formatting rules

Table	Key fields	Notes
voice_profiles	id, owner_id, label, style_vector, guide_text, sample_count	One per person; several for Edu Pro
usage_counters	user_id, day, words_scored, suggestions, uploads	Drives limits and the usage meter
risk_signals	id, user_id, signal_type, value, at	Inputs to the misuse model
risk_scores	id, user_id, score, level, reasons_json, at	Output and history
abuse_cases	id, user_id, level, status, reviewer_id, outcome, appeal_status	Operator queue
share_links	id, document_id, token_hash, expires_at, revoked_at, opened_count	Review links
feedback	id, suggestion_id, rating, reason, opted_in_training	Learning loop
model_versions	id, kind, name, metrics_json, status	Registry mirror
audit_log	id, actor_id, action, target, at, ip	Operator and security events

Relationships. A user owns folders and documents; a document has many versions; a version has a tree of nodes; text nodes have sentences; sentences have scores, suggestions and edits. Atoms attach to a document. Risk signals and scores attach to a user and feed the abuse cases.

Storage rules. Original and edited .docx files live in encrypted object storage, never in the database. Sentence text in the database is deleted together with the document. Deletion removes files, rows, embeddings and cached renders within 30 days, and backups age out within a documented window.

15. Security, privacy and compliance

People upload unpublished theses and client work, so trust in how files are handled is part of the product.

15.1 File handling

- Every upload is scanned for malware, macros are stripped from the working copy, and the file type is verified by content, not by extension.
- Files are processed in sandboxed workers with no outbound network access, and with CPU, memory and time limits, to contain zip bombs and malformed XML.
- Originals are encrypted at rest (AES-256) with per-tenant keys, and in transit (TLS 1.3).
- Documents are private by default; share links are random, expiring and revocable.

15.2 Access control

- Email and Google sign-in, optional two-factor authentication, and session management with a device list.
- Role-based access inside the operator console, with least privilege, just-in-time elevation and a full audit log.
- Staff cannot open a user's document text without a support ticket from that user, and every such access is logged and visible to the user.

15.3 Privacy

Topic	Policy
Training use	Off by default; opt-in only, with identifiers removed
Retention	Documents kept while the account is active; deleted 30 days after the user deletes them or the account closes
Third-party models	No document text is sent to outside AI providers; models run on our own infrastructure
Analytics	Event metrics without document content
User rights	Export all data, delete all data, see what is stored, correct profile data
Children	Service for users 16 and over, with age confirmed at sign-up

15.4 Compliance

GDPR and UK GDPR (data processing agreement, lawful basis, data subject requests, regional hosting option), Zimbabwe's Cyber and Data Protection Act and equivalent African data-protection laws where users are located, and CCPA for California. Payment

data stays with the processor (PCI DSS scope kept minimal). An institution-ready security pack (SOC 2 roadmap, penetration test summary, DPA) is prepared before the V2 institution plan.

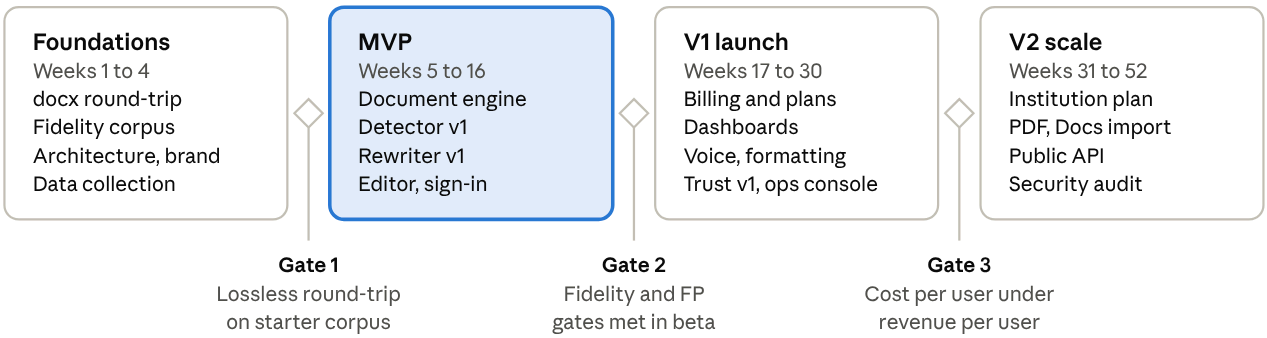
15.5 Responsible-use terms

The terms state that the product edits the user's own writing or writing they are authorised to edit, that detection scores are estimates and not proof, that results should not be used as the sole basis for an accusation of misconduct, and that users stay responsible for following their institution's rules on AI assistance. These terms are shown at sign-up and in the upload dialog.

16. Roadmap, team and costs

The plan builds the hardest and most defensible part first, the document engine, and adds each later layer only after a gate is met. Durations assume a small team and are proposals to be revised after phase 0.

Four phases, about 52 weeks, with a gate before each phase starts



roadmap · 4 phases, 3 gates

Gate 1 proves the engine can open and save real documents without damage; Gate 2 proves quality in a closed beta; Gate 3 proves the business model before institution work starts.

16.1 Phase deliverables and gates

Phase	Deliverables	Exit gate
Foundations	Parser and patcher prototype, sentence-to-run mapping, OMML and LaTeX round-trip spike, fidelity corpus (first 100 documents), data sourcing plan, architecture decisions, brand	Lossless round-trip on the starter corpus; GPU cost benchmark for one scored and rewritten document
MVP	Full document engine with atoms, detector v1, rewriter v1 with meaning checks, editor with click-to-suggest and auto, clean and tracked export, sign-in and basic accounts	Closed beta of about 50 users; fidelity corpus at 500 documents and passing; false-positive target met on the evaluation set
V1 launch	Plans and billing, both dashboards, usage meters, voice profiles, formatting profiles, trust service v1 and enforcement ladder, operator console, share links	Paying users; cost per user under revenue per user on each plan; abuse flag precision acceptable on reviewed cases
V2 scale	Institution plan with SSO, PDF and Google Docs import, public API, regional hosting, security audit	Repeatable sales to institutions; security pack ready

16.2 Proposed team

Role	MVP	V1	Notes
Product and design lead	1	1	Owns scope, UX, pricing tests
Document engineer (OOXML)	1	2	Hardest hire; Open XML experience matters most
Full-stack engineers	2	3	Editor, dashboards, API, billing
ML engineer	1	2	Detector, rewriter, serving
Data and evaluation lead (part time)	0.5	1	Corpora, annotation, human evaluation panel
DevOps and security (part time)	0.5	1	Infrastructure, GPUs, hardening

Role	MVP	V1	Notes
Support and trust reviewer	0	1	Abuse queue, appeals, billing queries

16.3 Cost drivers

Driver	How to estimate	How to control
GPU inference	Tokens per scored and rewritten document multiplied by documents per user per month	Small distilled models, caching, on-demand suggestions, daily caps
GPU training	Fine-tuning runs per quarter	LoRA instead of full fine-tuning, spot or reserved GPUs
Storage	Average document size multiplied by versions and retention	Delta storage for versions, 30-day deletion rule
Render workers	Pages rendered per export	Render only changed pages
Payment fees	Processor percentage plus fixed fee on each $5 charge	Annual plans, a minimum charge, local payment rails
People	Team table above	Hire the document engineer and ML engineer first

The fixed fee on a $5 payment is a real share of revenue with many processors, so the pay-per-paper option may need a small minimum fee or a bundle (for example three papers) after the first pricing test.

17. User manual

This chapter is written as the customer-facing help guide. It can be copied into the help centre once the product exists.

17.1 Getting started

1. Create an account with your email or Google, and verify your email.
2. Choose a plan: **Individual** if you are editing your own papers, or **Edu Professional** if you work on other people's papers (tutor, lecturer, editor, writing centre).
3. Add a payment method, or use your free trial page.

4. Optional but recommended: add two or three of your earlier documents to your voice profile.

17.2 Uploading a document

1. On the dashboard, drag a Word file (.docx) into the upload zone, or click to browse.
2. Confirm "This is my own paper" (Individual), or choose the client folder (Edu Pro).
3. Wait for the scan. You will see the number of pages, tables, equations and images found.
4. Your document opens in the editor, looking like your original.

17.3 Reading the scores

Mark	Meaning
No colour	Likely written by a person
Yellow	Uncertain, could be either
Orange or red	Likely machine-like phrasing
Grey with a lock	Locked or not scored (equations, quotes, references, very short text)

A score is an estimate based on writing patterns. It is not proof of who wrote the text, and a high score on a sentence can be a false alarm. Use the threshold slider at the top to show more or fewer marks.

17.4 Fixing a sentence

1. Click the underlined sentence.
2. In the right panel, read why it was flagged and look at the three suggestions.
3. Hover a suggestion to preview it in place. Click **Apply** to use it.
4. Not happy? Click **Auto** for the best option, or **Edit by hand** to type your own.
5. Click **Undo** at any time. Every change is saved in History.

17.5 Fixing a paragraph or the whole document

- Select a paragraph and click **Auto paragraph** to rewrite it sentence by sentence while keeping the flow.
- Click **Fix all above threshold** to build a review queue, then accept or skip each item.
- Use the strength control (light, moderate, full) to decide how far rewrites may go.

17.6 Equations

Equations appear as rendered formulas. Click one to open the LaTeX editor, change it, and watch the preview update. If an equation shows a lock, it contains a feature we keep untouched to protect it.

17.7 Protecting text

Click **Lock** on any sentence you do not want changed, for example a direct quote. Locked text is not rewritten and not scored. Click **Ignore** to mark a flagged sentence as reviewed.

17.8 Formatting

Open the Formatting tab, choose a style such as APA or Harvard, review the list of differences and the before and after preview, then click Apply. Your own formatting is only changed where it differs from the style you chose.

17.9 Exporting

1. Click Export.
2. Choose **Clean** (your edits applied) or **Tracked changes** (every edit shown for review in Word).
3. Read the fidelity report. It lists what changed and confirms that the rest is identical.
4. Download your .docx.

17.10 For Edu Professionals

- **Folders.** Create a folder per client, student or class. Move documents between folders by dragging.
- **Board.** Move documents through Uploaded, In review, Returned and Closed, and set deadlines.
- **Returning work.** Send the edited file to the owner, or create an expiring read-only review link.
- **Style profiles.** Save a house style once and apply it to a whole folder.
- **Several voices.** Keep one voice profile per client. Only add a client's samples with their permission.
- **Batch.** Run scoring or formatting across a folder and review the results in the queue.

17.11 Plans, limits and billing

The usage meter on your dashboard shows today's words and suggestions and your active papers. When you reach a limit you can wait until midnight, buy a single paper for $5, or upgrade. You can change or cancel your plan at any time under Billing. If you work on other

people's papers on an Individual plan, we will invite you to upgrade to Edu Pro.

17.12 Troubleshooting

Problem	What to do
Upload fails	Check the file is .docx (not .doc), under 50 MB and not password-protected
Formatting looks different in the editor	Report it with the "Looks wrong" button; we compare against your original
An equation will not render	Keep it as is (it is locked) or retype it in the LaTeX box
A suggestion changed my meaning	Click Reject and mark "Changed my meaning"; this trains the system
I hit my daily limit	Wait for the reset, buy a single paper, or upgrade
I was asked to confirm ownership	Confirm if the paper is yours, or switch to Edu Pro for other people's papers
I think my score is wrong	Scores are estimates; use Ignore, and send feedback so we can improve

18. Developer manual

This chapter is the engineering guide for the team that builds the platform.

18.1 Repository layout (monorepo)

Path	Contents
apps/web	Next.js frontend (editor, dashboards, marketing, billing pages)
apps/api	Backend API (auth, plans, documents, usage, billing webhooks)
apps/ops	Operator console
services/docx-engine	Parser, sentence-to-run mapper, patcher, exporter, OMML and LaTeX converters

Path	Contents
services/render	LibreOffice render workers and visual diff
services/detect	Detection ensemble serving
services/rewrite	Rewriter serving, meaning checks, constraint checks
services/trust	Misuse signals, risk scoring, enforcement jobs
ml/	Training code, data pipelines, evaluation suites, model cards
packages/schema	Shared types and the API contract
packages/ui	Shared design system and components
tests/fidelity	The 500-plus document regression corpus and its runner
infra/	Terraform, Helm charts, CI configuration
docs/	This document, architecture decision records, runbooks

18.2 Local setup

1. Install Node 20 or later, Python 3.11 or later, Docker, and a package manager (pnpm).
2. Start dependencies with Docker Compose: Postgres, Redis, MinIO (local object storage).
3. Run database migrations, then seed a test user, plans and a sample document.
4. Start the web app, API and docx-engine. The detection and rewrite services run in "stub" mode with small models on CPU for local development.
5. Open the sample document and confirm the editor shows scores and suggestions.

18.3 Conventions

- TypeScript on the frontend and API, Python for the docx engine and all ML services.
- One API contract (OpenAPI) generated into typed clients; breaking changes need a version bump.
- Every document edit is an event (accept, reject, undo) and the document state is rebuilt from the event log, which gives history and restore for free.

- No service reads a document's text except through the docx-engine API, so protected atoms are handled in one place.
- Feature flags for every new model and every limit.
- Architecture decision records (short Markdown files) for any choice that is hard to reverse.

18.4 Testing

Layer	Tests
docx-engine	Property tests (parse, no-op patch, export gives identical XML for untouched nodes); the fidelity corpus; round-trip tests for equations
Detection	Frozen evaluation set; false-positive gates by writer group; paraphrase-robustness set
Rewrite	Meaning-check and constraint pass rates; golden examples; human preference sampling
Trust	Replay of labelled historical cases; precision per level
API	Contract tests, plan-limit tests, billing webhook simulations
Frontend	Component tests, end-to-end editor flows with Playwright, accessibility checks
Non-functional	Load tests for upload bursts, GPU queue soak tests, security scans

18.5 Release process

1. Pull request with tests and a model card or ADR if relevant.
2. CI runs unit tests, the fidelity corpus and the evaluation gates.
3. Deploy to staging, run a smoke suite with real sample documents.
4. Canary to 5 percent of traffic, watch the quality monitors (acceptance, meaning failures, fidelity errors, latency).
5. Roll out fully, or roll back automatically if a gate fails.

18.6 Observability and operations

- Structured logs, traces and metrics (OpenTelemetry) with a trace ID from upload to export.
- Alerts on queue depth, GPU saturation, fidelity failures, payment failures and cost per user.
- On-call rotation, runbooks for each service, and a post-incident review for every severity-one incident.

18.7 API surface (v1, internal)

Endpoint	Purpose
POST /documents	Upload and start the scan
GET /documents/:id/structure	Structure map with sentence IDs
GET /documents/:id/scores	Streamed scores by paragraph
POST /sentences/:id/suggest	Get suggestions with strategy, tone and strength
POST /sentences/:id/apply	Apply a suggestion or hand edit
POST /paragraphs/:id/auto	Auto-rearrange a paragraph
POST /documents/:id/format	Preview or apply a style profile
POST /documents/:id/export	Create clean or tracked export
GET /usage	Current meter and limits
POST /voice-profiles	Create from samples
POST /share-links	Create review link
POST /billing/webhook	Processor events

19. Risks and decisions needed

The three risks that can sink the product are broken formatting, wrongly accused users, and model cost above the $5 price. Each has a named mitigation.

19.1 Risk register

Risk	Likelihood	Impact	Mitigation
Export breaks formatting or loses content	Medium	Critical	Patch-in-place design, fidelity corpus gate on every release, visual diff before export, original always restorable
False positives lead to a wrongly accused user	Medium	High	Confidence bands, calibration by writer group, plain language that scores are estimates, no verdicts
Detectors change and the product is judged on beating them	High	High	Position on voice and clarity, platform intelligence layers (section 11), rapid retraining cadence
GPU cost per user exceeds $5 or $15	Medium	High	Small models for the common path, distillation, quantisation, batching, daily caps, per-plan cost dashboard
Equation round-trip fails on complex formulas	High	Medium	Lock-and-keep fallback, regression suite, LaTeX edit only where round-trip is proven
Misuse model wrongly flags honest users	Medium	Medium	Graduated ladder, courteous nudges first, human review at level 4, appeals, false-flag metric
Academic-integrity backlash or app-store and payment-processor policy issues	Medium	High	Responsible-use terms, no "undetectable" claims, focus on own-writing improvement and clarity, legal review of marketing copy
Data breach of unpublished papers	Low	Critical	Encryption, sandboxed workers, least privilege, short retention, penetration tests
Open-model licence changes	Low	Medium	Keep two candidate models per job; licence review at each model adoption

Risk	Likelihood	Impact	Mitigation
Payment coverage in target markets	Medium	Medium	Multiple processors, local options such as mobile money

19.2 Decisions needed from you

- [ ] Confirm the pricing reading in section 9.5: $5 per paper as a separate option, and $5 per month for one person's own papers.
- [ ] Choose hard daily caps or only abuse throttling for the Individual plan.
- [ ] Set the active-papers cap for Individual and Edu Pro.
- [ ] Decide the launch market and currency (this affects processor choice and compliance).
- [ ] Decide whether the detector is built first, or the document engine plus rewriter first (recommended: document engine first, because it is the hardest to get right and the most defensible).
- [ ] Decide the brand and domain name.
- [ ] Decide whether to host on your own GPUs or rent cloud GPUs at launch (recommended: rent, then reassess at steady volume).
- [ ] Confirm who reviews abuse cases at launch (you, a support hire, or a contractor).